

Turn On LED Light

Turn On LED Light

1. Learning Objectives
2. Hardware Setup
3. Experiment Steps
 1. Open SYSCONFIG Configuration Tool
 2. Parameter Settings
 - 2.1 Set Port Parameters
 - 2.2 Set Pin Parameters
 - 2.3 Save and Update Configuration File
 3. Write Program
 4. Compile
4. Program Analysis
5. Experimental Phenomena

1. Learning Objectives

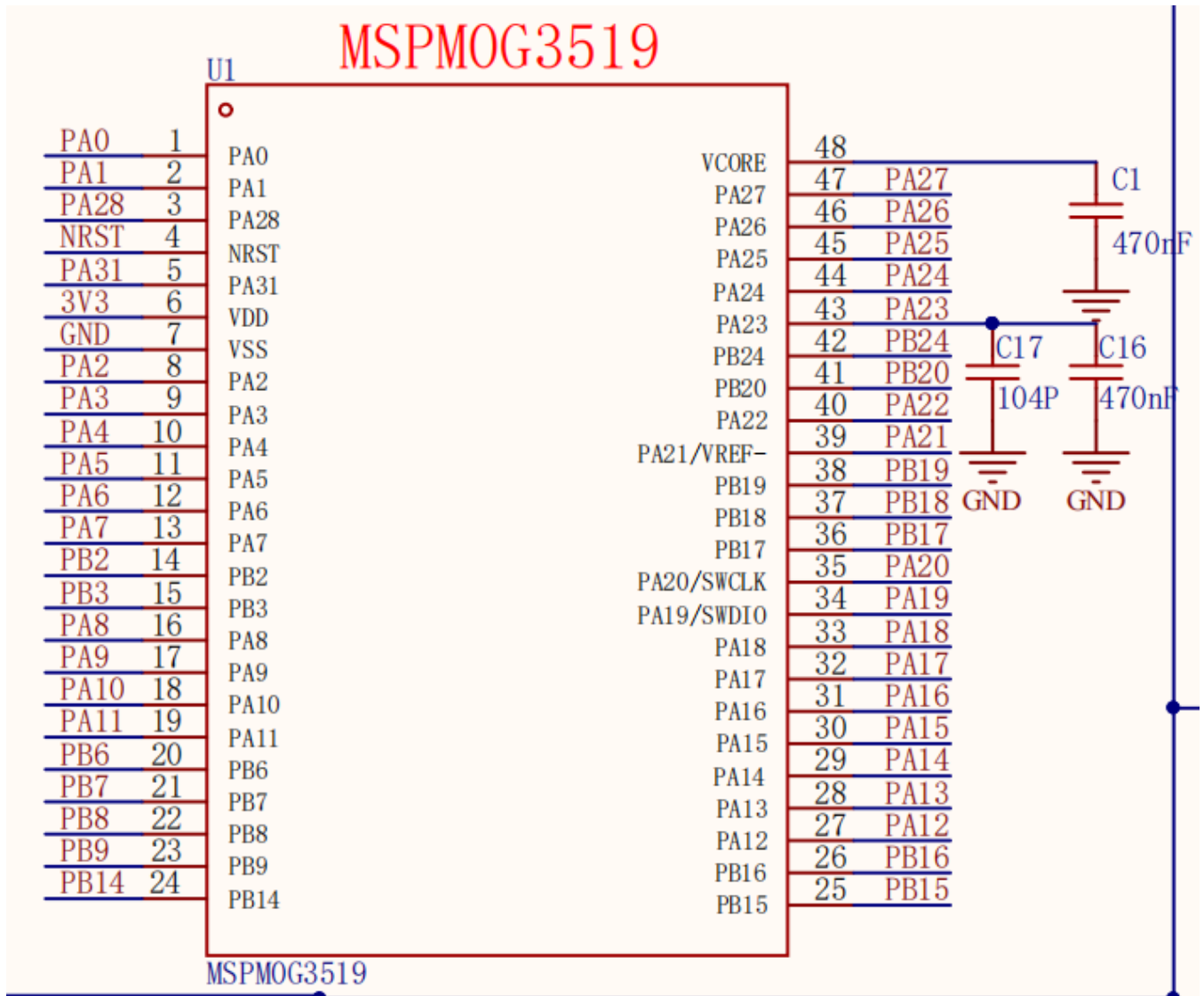
1. Learn the basic use of MSPM0G3519 motherboard pins.
2. Understand how to control the onboard LED light.

2. Hardware Setup

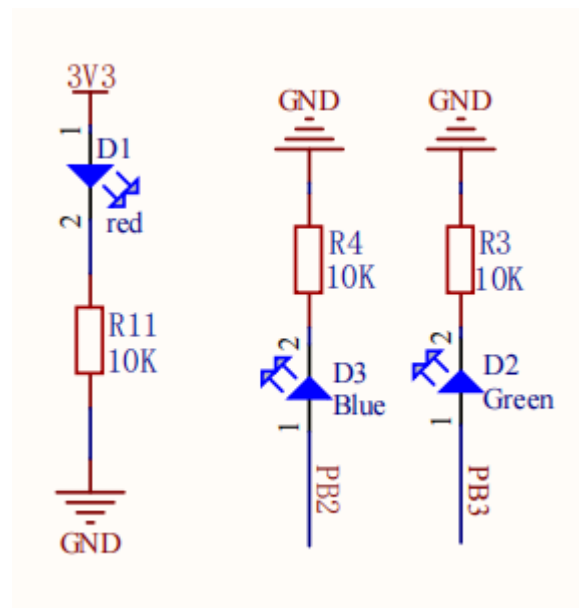
This course does not require additional hardware. You can directly use the onboard LED light on the MSPM0G3519 motherboard.

We have set up two LED user lights on the motherboard. Users can customize their functions. The following uses **LED1** as an example.

MSPM0G3519 Main Control Diagram



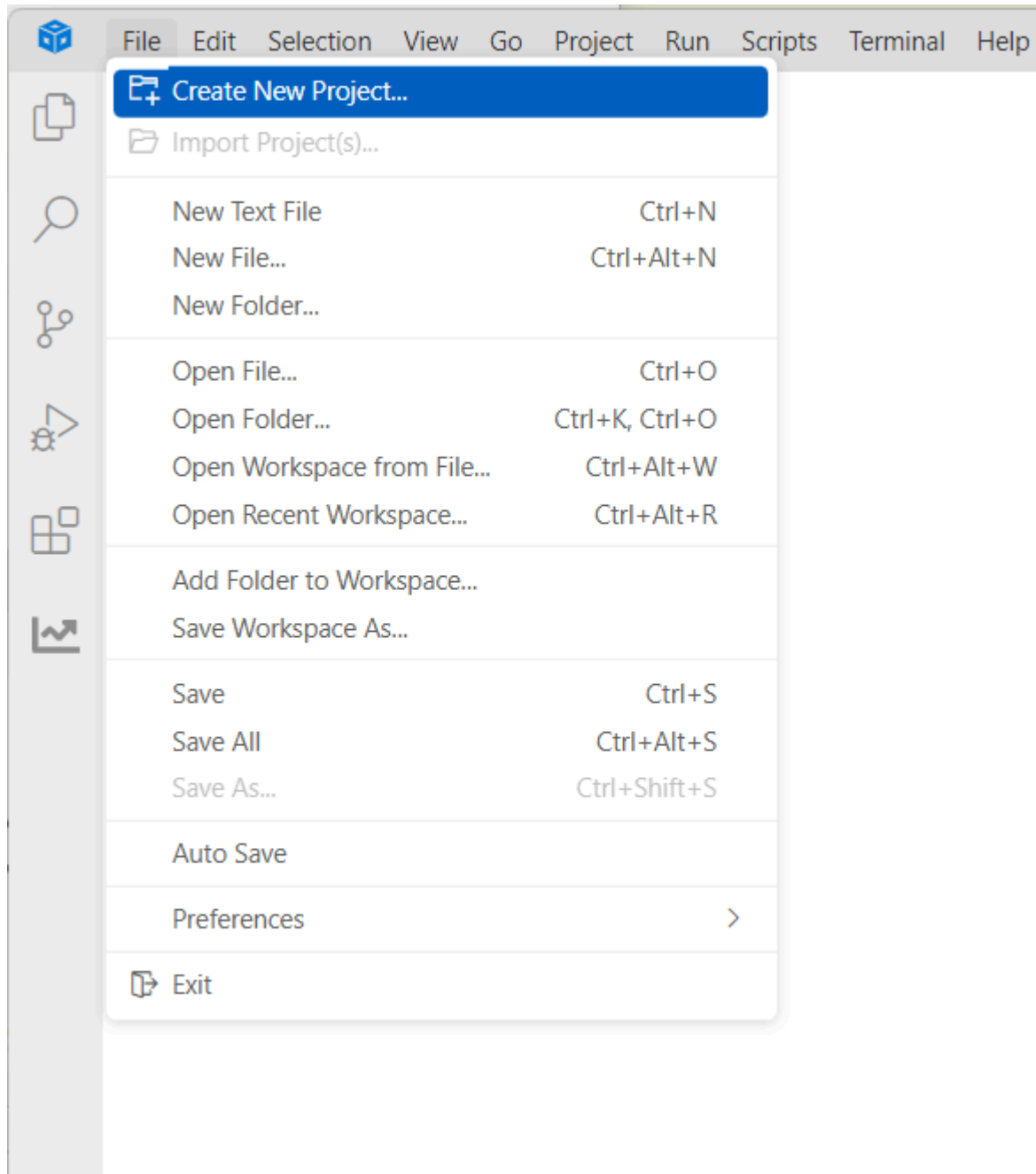
LED Part Schematic

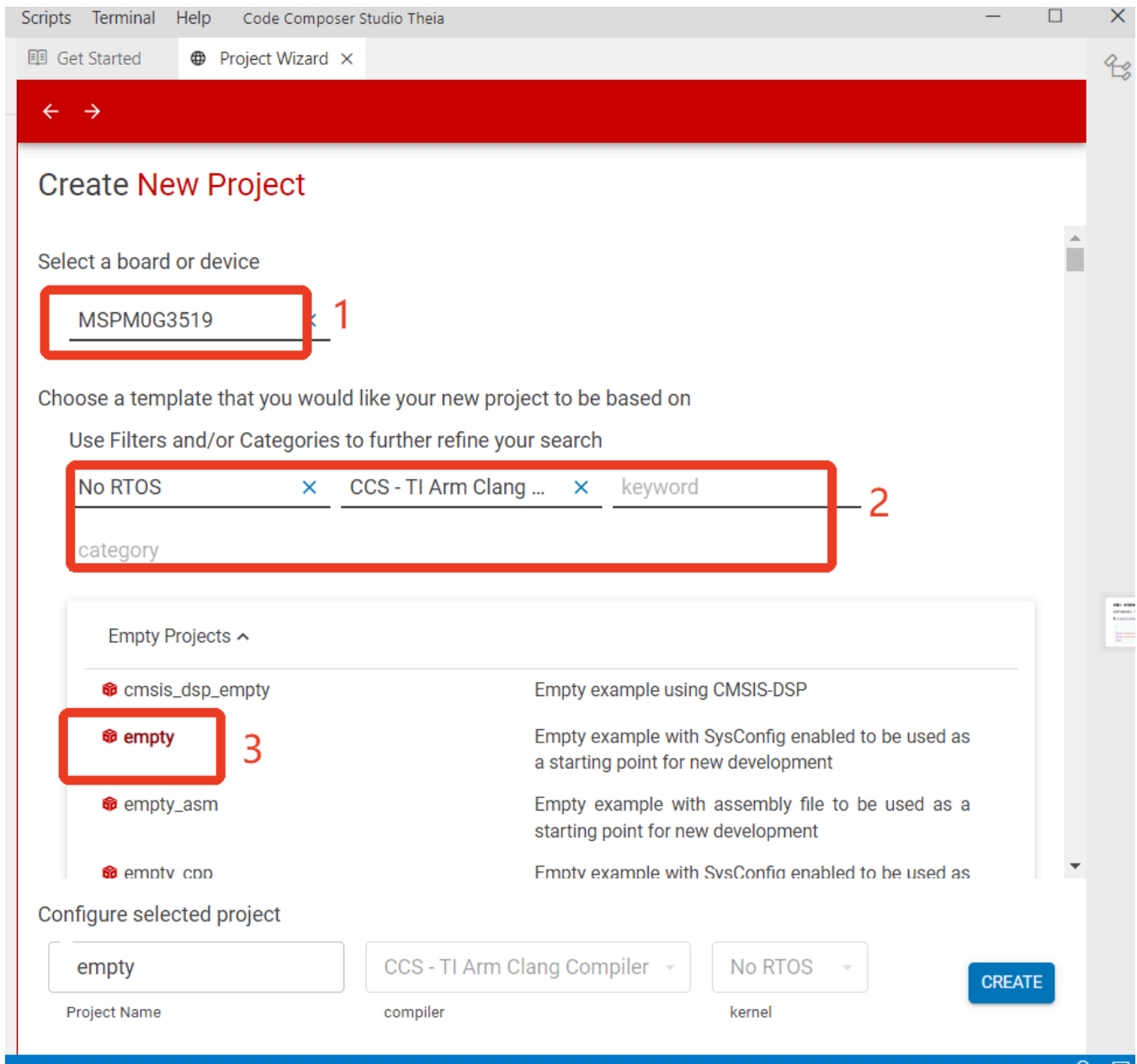


3. Experiment Steps

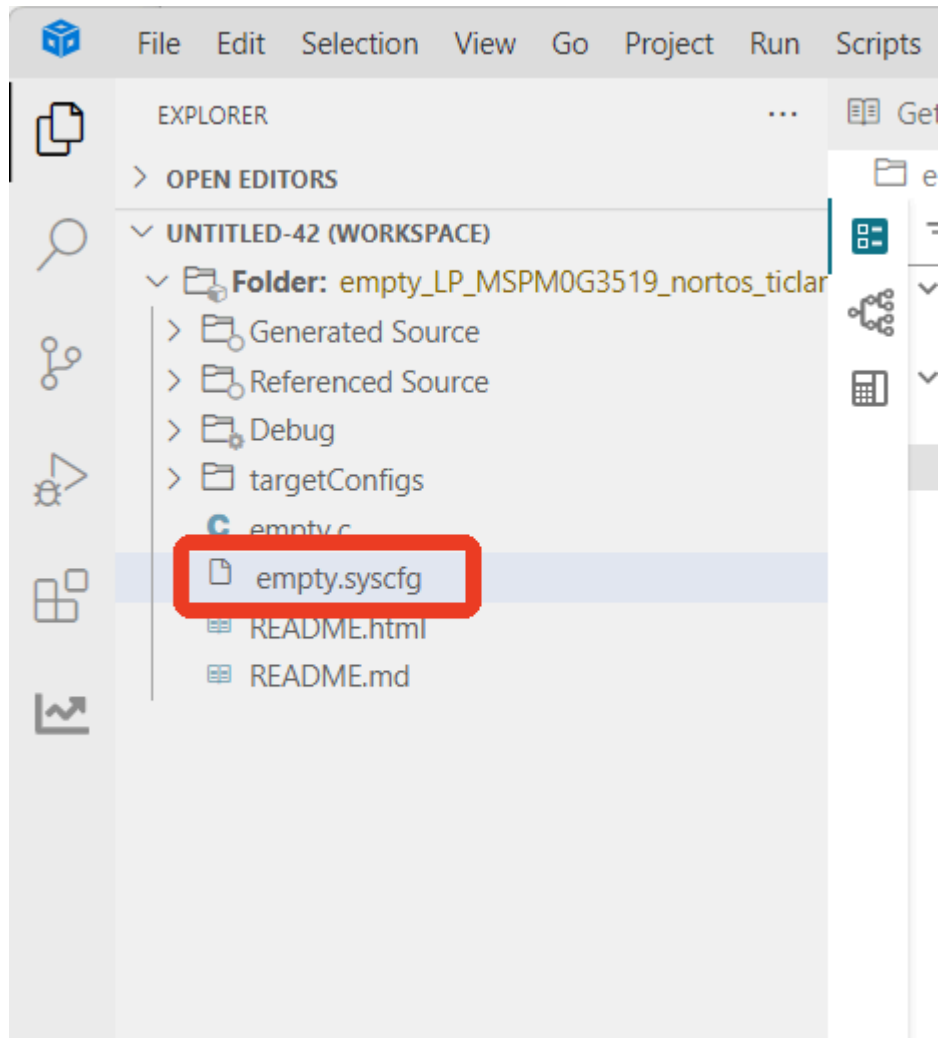
1. Open SYSCONFIG Configuration Tool

Create a new empty project empty in CCS.



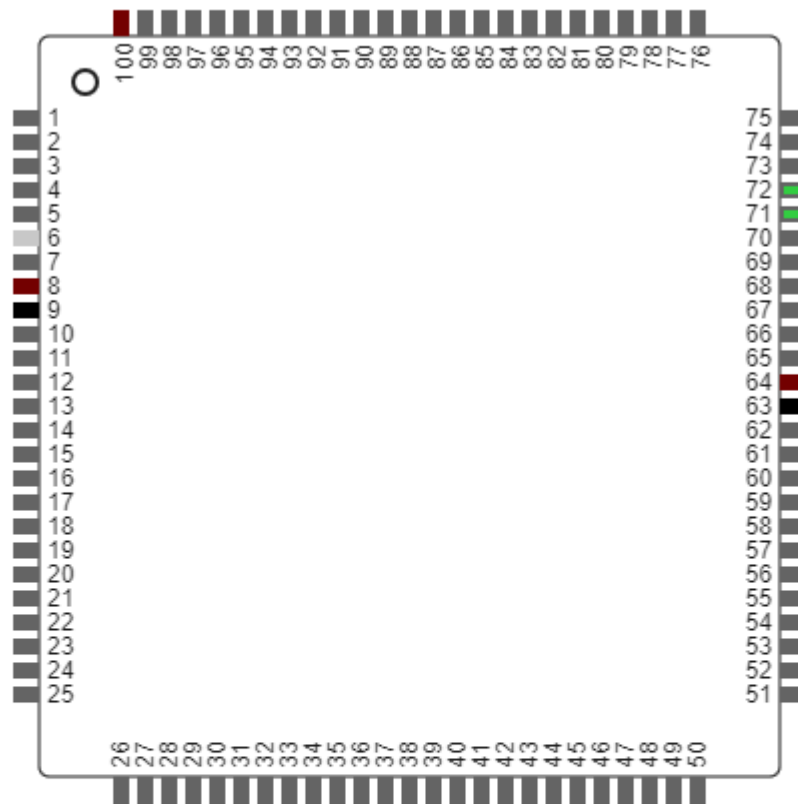


Find and open the empty.syscfg file in the left workspace of CCS.



In SYSCONFIG, Modify chip pin package

MSPM0G3519 (Device)
LQFP-100(PZ) (Package)



SWITCH

Pin Available



Pin Assigned



Warning



Power



Ground



Fixed (N/A)



⚠ Migrate Settings

This will migrate the current configuration to the board or device selected below. Any incompatibilities will be flagged as errors.

The migration can be undone by using ctrl + z or the history view. Once satisfied with the conversion, saving the changes will initiate the migration of the underlying project which cannot be undone.

See [MSPM0 SDK CCS IDE Guide](#) for extended details on Migrating Between MSPM0 Derivatives.

Setting	Current Value	New Value
Board		None
Device	MSPM0G3519	MSPM0G3519
Package	LQFP-100(PZ)	LQFP-100(PZ)
Lock Resource Allocation		

LQFP-48(PT)
LQFP-64(PM)
LQFP-64(PTE)
LQFP-80(PN)
LQFP-100(PZ)
VQFN-32(RHB)
VQFN-48(RGZ)

ANCELCONFIRM

select MCU peripherals on the left, find the GPIO option and click to enter. Click ADD in GPIO to add a GPIO group.

The screenshot shows the CCS IDE interface for configuring the MSPM0G3519 device. The left sidebar displays the project tree with the following structure:

- PROJECT CONFIGURATION (1)
 - Project Configur... 1/1
- MSPM0 DRIVER LIBRARY (7)
 - SYSTEM (10)
 - Board 1/1
 - Configuration NVM
 - GPIO**
 - IOCON
 - MATHACL
 - RTC_B
 - SYSTICK
 - SYSTICK
 - WWD
 - ANALOG (4)
 - ADC12
 - COMP
 - DAC12
 - VREF
 - COMMUNICATIONS (6)
 - I2C
 - I2C - SMBUS
 - MCAN
 - SPI
 - UART
 - UART - LIN
 - TIMERS (6)
 - TIMER
 - TIMER - CAPTURE
 - TIMER - COMPARE
 - TIMER - PWM
 - TIMER - QE
 - Timer Fault
 - SECURITY (3)
 - AESADV
 - Security Configurator

The main area displays the GPIO configuration options, including the 'ADD' button (highlighted with a red box) and the 'REMOVE ALL' button. The GPIO configuration options are:

- Put GPIO's into groups
- Configure digital IOMUX features
- Configure I/O based on Interrupts/Events
- Configure GPIO pins in low-power modes using FaskWake

Under Advanced Configuration users can configure glitch filters, clock timeout detection, etc.

For Global Pin Configuration, see the Board module.

The diagram shows the GPIO configuration options (DOUT[SET/CLR/TGL], DOE[SET/CLR], DIN, POLARITY, CTL, FASTWAKE, FILTEREN, DMAMASK) and the Event Publisher 2:0. The CPU and Event blocks are also shown.

The right sidebar shows the 'Generated Files' list and the 'MSPM0G3519 (Device)' pin configuration diagram.

The 'Generated Files' list includes:

- device_linker.cmd (Linker Command)
- device.opt (Compiler Options)
- device.cmd.genlibs (Library Includes)
- ti_msp_dl_config.c (MSPM0 Driver Library)
- ti_msp_dl_config.h (MSPM0 Driver Library)
- empty.syscfg (Configuration Script)

The 'MSPM0G3519 (Device)' pin configuration diagram shows the pin numbers (1-17) and the package type (LQFP-100(PZ)).

2. Parameter Settings

2.1 Set Port Parameters

Configure GPIO parameters and name the pin LED1.

From the LED part schematic, LED1 is on port PB2, so set Port to PORTB.

GPIO (1 Added)

+

ADD

REMOVE ALL

✓

LED1

Name	LED1
Port	PORTB
Port Segment	Any

Parameter description:

Name: Custom name for the GPIO instance. By default, the name starts with the numeric suffix "0". You can customize the name to reflect the module's purpose (for example, naming the GPIO "LED1" so you know this pin is specifically for controlling LED1).

Port: The GPIO instance's port. LED is connected to GPIOB2 pin, so only PORTB can be selected.

Port Segment: Set port pull-up/down resistor. Note this is port pull-up/down, which sets the entire GPIOB port.

2.2 Set Pin Parameters

Set the pin to output mode, set the pin to output low level by default, set the pin to pull-down mode, set the pin to GPIOB_2, do not enable interrupt events.

Group Pins
^

1 added

+ ADD
REMOVE ALL

✓ PIN_2
🗑

Name	PIN_2
Direction	Output ▼
Initial Value	Cleared ▼
IO Structure	Any ▼

Digital IOMUX Features
^

Internal Resistor	Pull-Down Resistor ▼
Invert	Disabled ▼
Drive Strength Control	Low ▼
High-Impedance	Disabled ▼

Assigned Port	PORTB ▼
Assigned Port Segment	Any ▼
Assigned Pin	2

Interrupts/Events
^

Event Subscribing Channel	Disabled (0) ▼
Output Policy	Bit will be Set ▼

LaunchPad-Specific Pin	No Shortcut Used ▼
------------------------	---------------------------------

Parameter description:

Name: User-defined pin name, set to PIN_2.

Direction: Set pin mode, input and output. Here we are controlling the LED light, so select output mode.

Initial Value: Set the initial state of the pin, can only be set when configured as output mode. There are two options, clear and set. Clear means output low level, set means output high level.

IO Structure: Set IO structure. There are multiple options: Default (Any), Standard, Wake (Standard with Wake), High-Speed, and 5V Tolerant Open Drain.

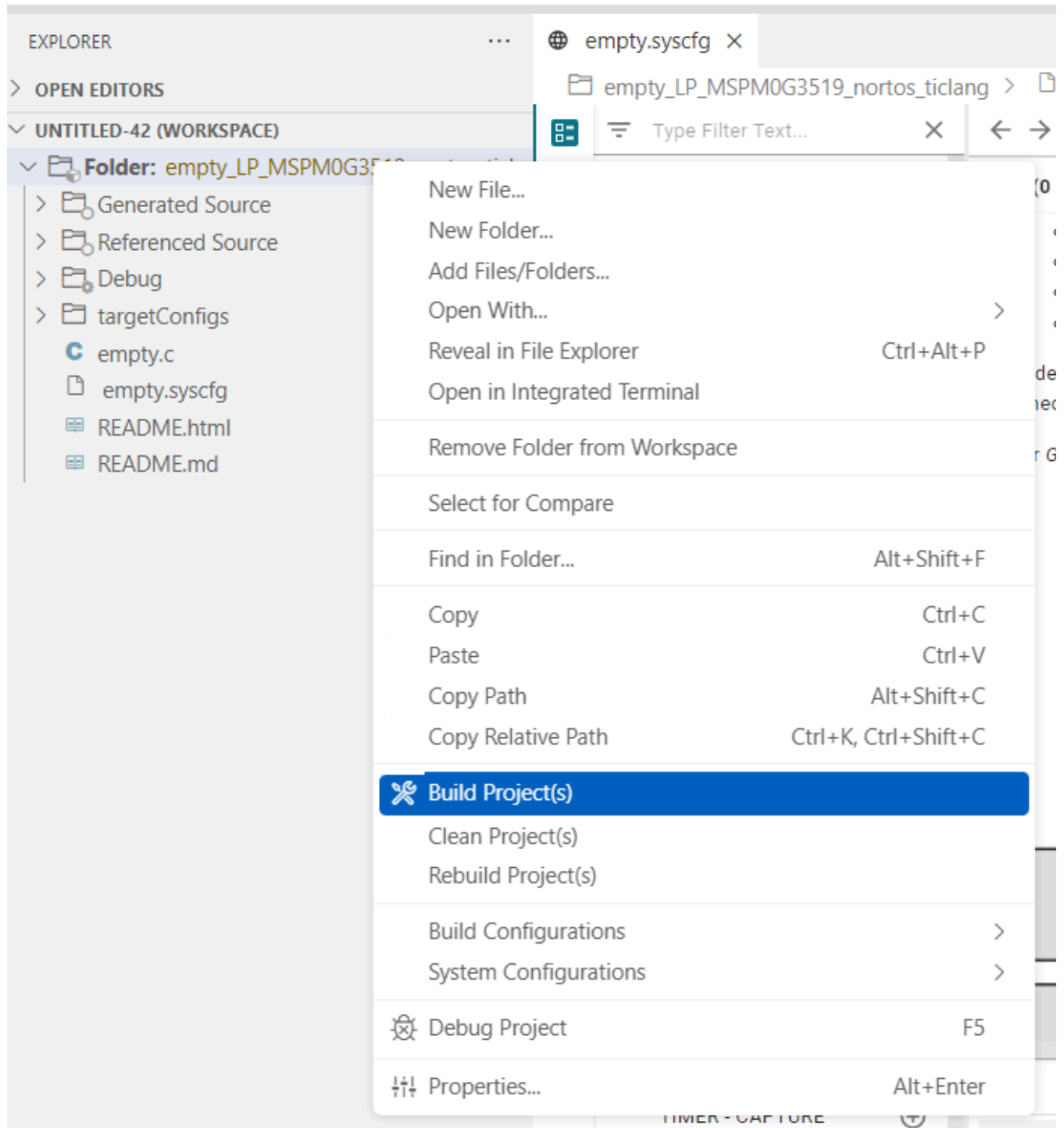
Internal Resistor: Set pin pull-up/down resistor. There are three options: none, pull-up, and pull-down resistor. Based on the LED connection method, pull-down resistor is selected here.

Assigned Pin: Set pin number. Enter the corresponding pin number for whichever pin you want to control. For example, the LED on the development board is connected to GPIOB2, so enter 2.

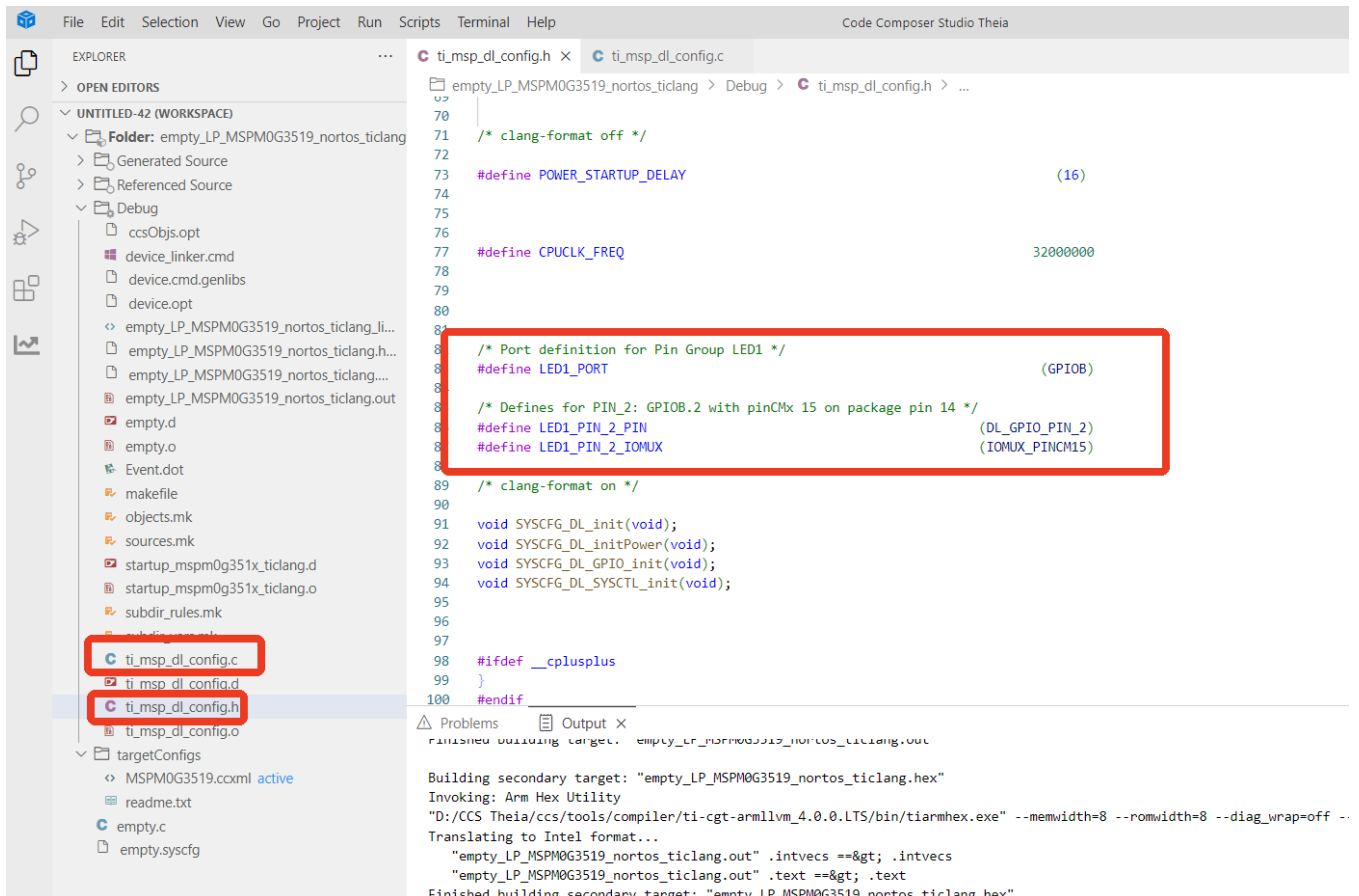
2.3 Save and Update Configuration File

Save the configuration in the .syscfg file using the shortcut key Ctrl+S.

After saving, compile.



After compilation, you can see that two more files have been added under the Debug folder: `ti_msp_d1_config.c` and `ti_msp_d1_config.h`. These two files are the peripheral low-level driver configuration code generated based on the .syscfg file we just saved.



3. Write Program

In the empty.c file, write the following code

```
#include "ti_msp_dl_config.h"

//Custom delay (not precise)
void delay_ms(unsigned int ms)
{
    unsigned int i, j;
    // The number of nested loops below is roughly calculated based on the master control
    frequency and the instruction cycle generated by the compiler, and needs to be adjusted
    through actual testing to achieve the required delay.
    for (i = 0; i < ms; i++)
    {
        for (j = 0; j < 8000; j++)
        {
            // Perform only one operation that is simple enough to predict its execution
            time
            __asm__("nop"); // "nop" stands for "no operation", which on most architectures
            consumes one or a few clock cycles
        }
    }
}

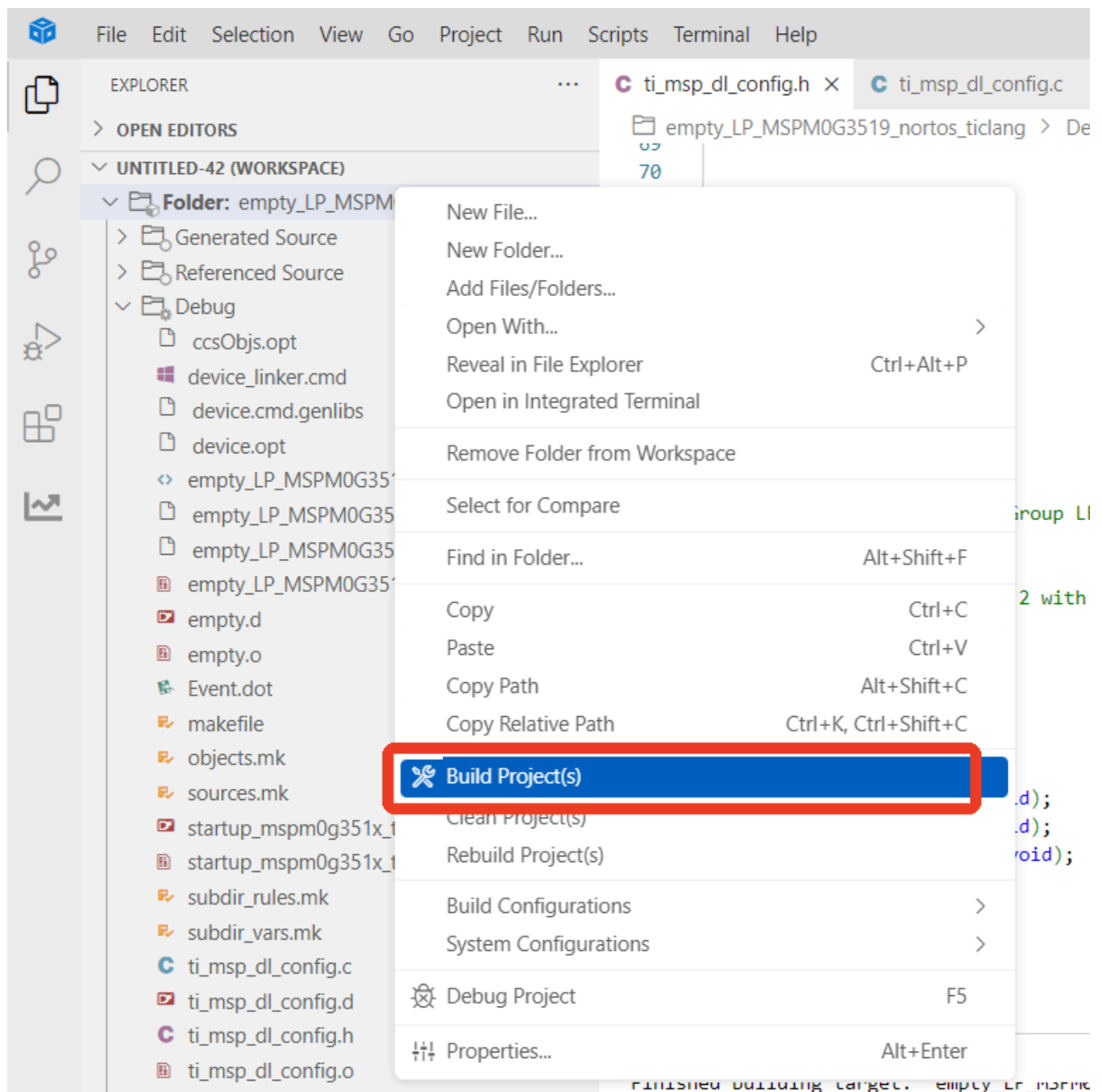
int main(void)
{
    SYSCFG_DL_init();
}
```

```

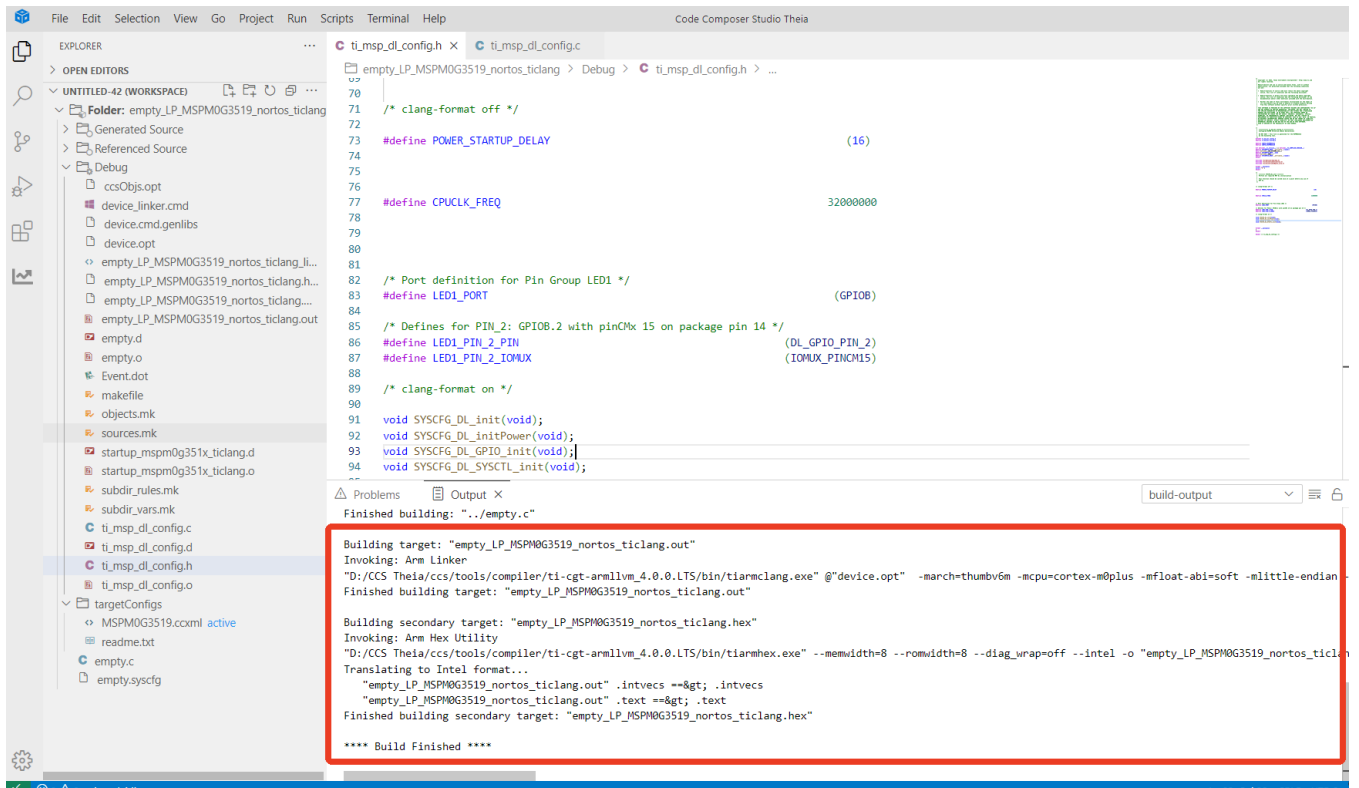
while (1)
{
    DL_GPIO_clearPins(LED1_PORT,LED1_PIN_2_PIN); //Output low level
    delay_ms(1000); //Delay about 1s
    DL_GPIO_setPins(LED1_PORT,LED1_PIN_2_PIN); //Output high level
    delay_ms(1000); //Delay about 1s
}
}

```

4. Compile



It shows compilation was successful here. You can now download the program to the development board.



4. Program Analysis

- dl_gpio.h



_STATIC_INLINE void DL_GPIO_setPins(GPIO_Regs* gpio, uint32_t pins): This function controls pins to output high level.

__STATIC_INLINE void DL_GPIO_clearPins(GPIO_Regs* gpio, uint32_t pins): This function controls pins to output low level.

- empty.c

```
1  #include "ti_msp_dl_config.h"
2
3  //自定义延时 (不精确) Custom delay (not precise)
4  void delay_ms(unsigned int ms)
5  {
6      unsigned int i, j;
7      // 下面的嵌套循环的次数是根据主控频率和编译器生成的指令周期大致计算出来的,
8      // 需要通过实际测试调整来达到所需的延时。
9      // The number of nested loops below is roughly calculated based on the master control frequency and the instruction cycle gener
10     for (i = 0; i < ms; i++)
11     {
12         for (j = 0; j < 8000; j++)
13         {
14             // 仅执行一个足够简单以致于可以预测其执行时间的操作
15             // Perform only one operation that is simple enough to predict its execution time
16             __asm__("nop"); // "nop" 代表“无操作”, 在大多数架构中, 这会消耗一个或几个时钟周期 "nop" stands for "no operation", which on
17         }
18     }
19 }
20
21 int main(void)
22 {
23     SYSCFG_DL_init();
24     while (1)
25     {
26         DL_GPIO_clearPins(LED1_PORT, LED1_PIN_2_PIN); //输出低电平 Output low level
27         delay_ms(1000); //延时大概1S Delay about 1S
28         DL_GPIO_setPins(LED1_PORT, LED1_PIN_2_PIN); //输出高电平 Output high level
29         delay_ms(1000); //延时大概1S Delay about 1S
30     }
31 }
```

Uses imprecise delay. Set PB2 to output low level, delay about 1s, then output high level, then delay about 1s. Repeat this process.

5. Experimental Phenomena

After the program is downloaded, you can see LED1 on the MSPM0 development board light up and blink approximately every 1 second.

